INFORME DE EVIDENCIAS TÉCNICAS — SESIÓN 11

Seguridad Frontend SPA: Autenticación con Keycloak OIDC (Authorization Code + PKCE), Guards RBAC e Interceptor con Refresco

1. Datos del Estudiante

Nombre del estudiante:	Faijo Calisaya Helio Paul (Código: 202110292)
Equipo y Módulo:	Equipo 05 — Módulo Mayorista (Centros de Acopio y Liquidaciones)
Sesión Evaluada:	S11 - Seguridad Frontend: Autenticación con Keycloak y Rutas Protegidas
Rol o aporte realizado:	Implementación integral del flujo Authorization Code + PKCE en Angular 19, AuthService con tokens en memoria, interceptor HTTP con refresco transparente ante 401 y guards de navegación RBAC.
Link de GitHub del proyecto:	https://github.com/helionelio900-hub/SysProyec_Acopus (Rama: main)

2. Alcance y Propósito de la Actividad (S11)

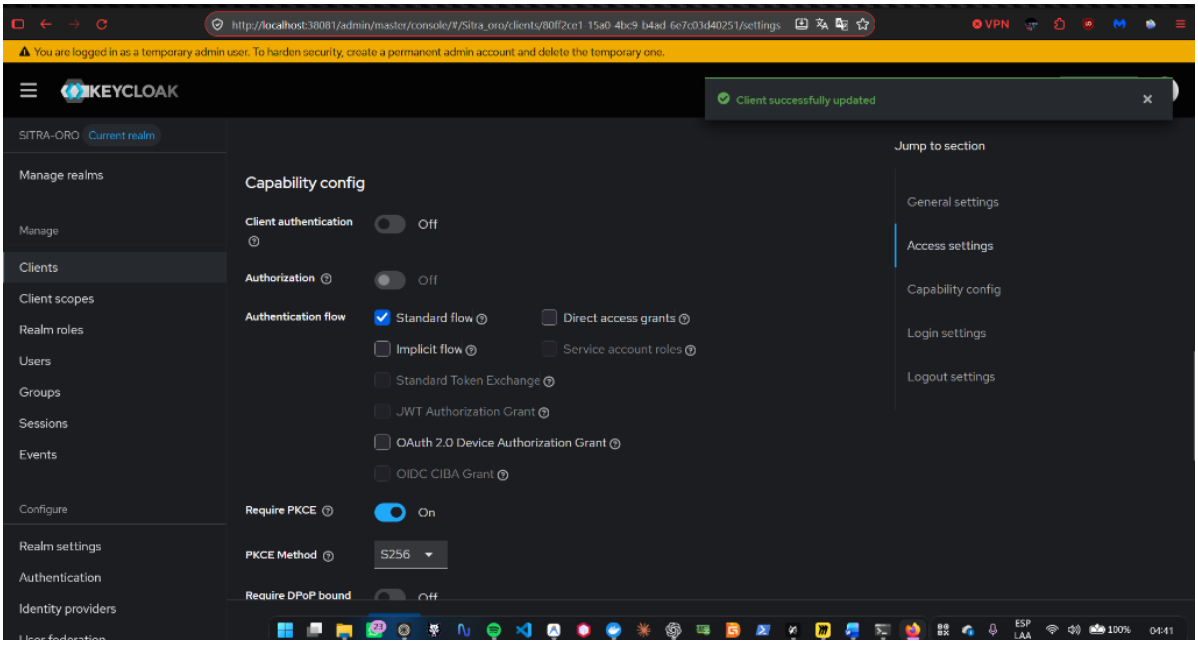
El presente informe demuestra la integración de la Single Page Application (SPA) SITRA-ORO (desarrollada con Angular 19 Standalone) contra un Identity Provider (IdP) OpenID Connect real — **Keycloak 26** en el Realm propio **Sitra_oro** (puerto 38081). Se implementó el estándar de seguridad industrial **Authorization Code Flow con PKCE obligatorio (RFC 7636)**, descartando el intercambio de credenciales directas (ROPC) y asegurando que las credenciales criptográficas (accessToken y refreshToken) **residan en memoria volátil de Angular Signals, sin persistirse en localStorage**, mitigando de raíz vectores de robo de tokens tipo Magecart/XSS.

3. Evidencia Técnica (Bloques de la Rúbrica de Evaluación)

Bloque 1: Login con Authorization Code + PKCE (Keycloak Real)

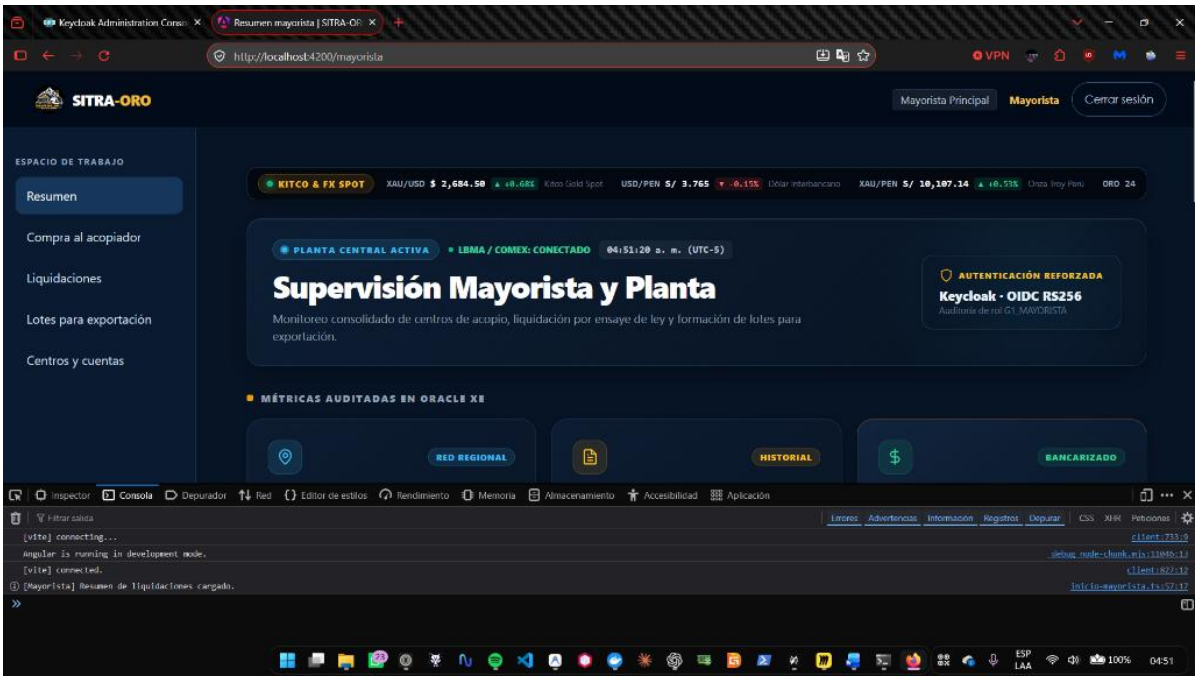
El cliente público **sitra-oro-frontend** fue configurado en Keycloak con Client Authentication desactivado (cliente público para SPA), **Direct Access Grants DESACTIVADO** y **Proof Key for Code Exchange (PKCE) establecido obligatoriamente en S256**. El flujo inicia en la SPA generando en tiempo de ejecución un code_verifier criptográfico y su hash SHA-256 (code_challenge), redirigiendo a Keycloak. Tras la autenticación, Keycloak redirige a /auth/callback retornando el code y state, donde la SPA efectúa el intercambio por tokens en memoria.

CAPTURA 1: Configuración del Cliente en Keycloak (PKCE S256 Obligatorio y Direct Access Grants OFF)



Evidencia técnica: Consola de administración de Keycloak (puerto 38081) -> Realm Sitra_oro -> Clients -> sitra-oro-frontend -> Pestaña Capability config: Standard Flow activo, Direct Access Grants desactivado, y Proof Key for Code Exchange (PKCE) en S256 obligatorio (reloj y usuario visibles).

CAPTURA 2: Login Real Completado de Punta a Punta con Redirección OIDC y PKCE

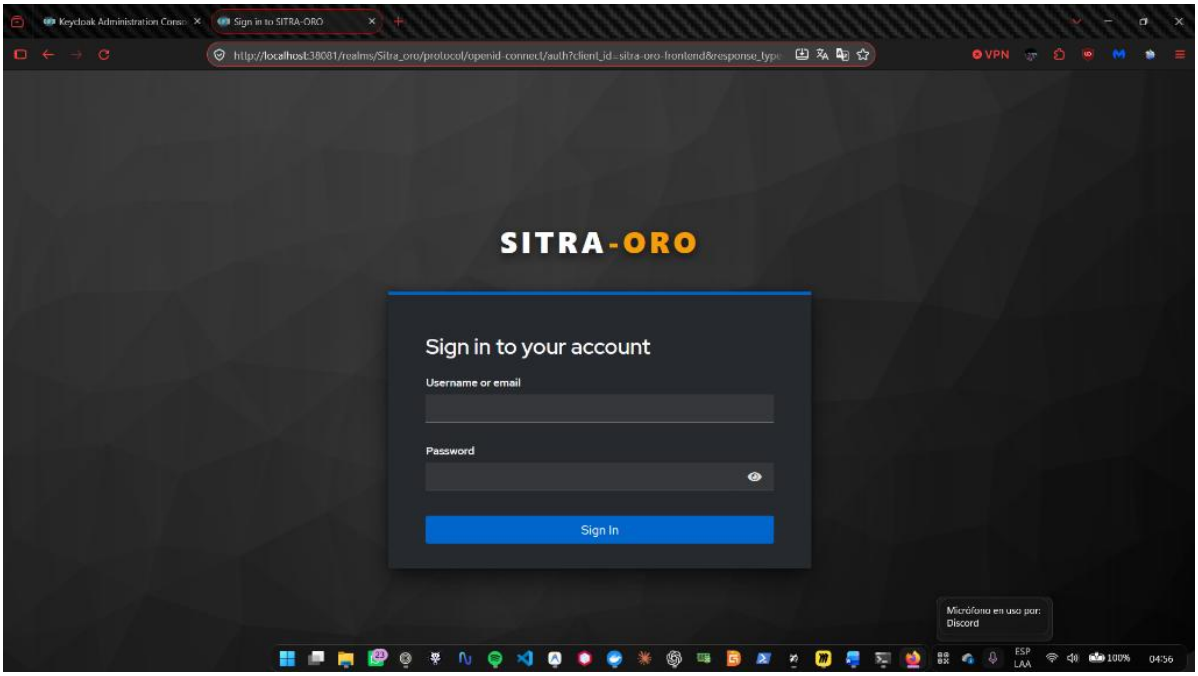


Evidencia técnica: Flujo de autenticación completado desde http://localhost:4200/ingresar hacia Keycloak y retorno exitoso mediante Authorization Code + PKCE. La SPA decodifica el token en memoria, reconoce la autoridad de rol G1_MAYORISTA y proyecta el panel de Supervisión Mayorista y Planta en /mayorista (reloj y usuario visibles).

Bloque 2: Guards de Autenticación y Rol (rolGuard)

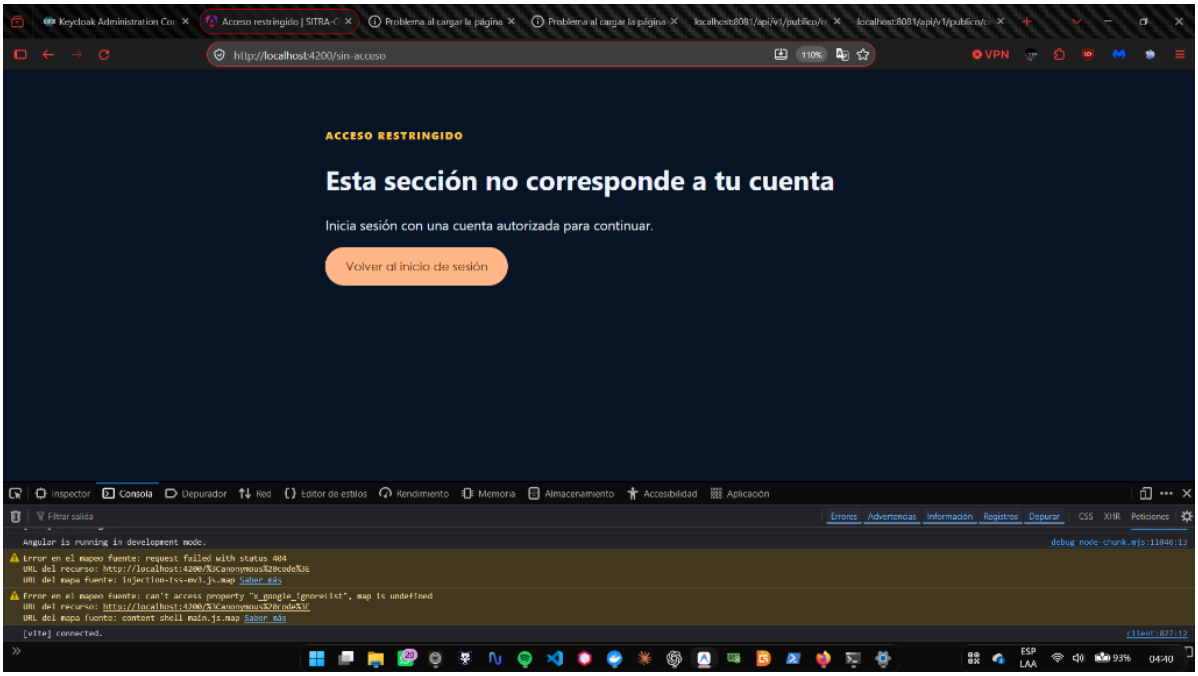
El guard funcional **rolGuard (CanActivateFn)** intercepta cada transición de ruta. Se verificaron los dos comportamientos de seguridad perimetral exigidos: (1) Intento de navegación sin sesión hacia una ruta privada redirige de inmediato al login de Keycloak preservando el destino original; y (2) Intento de acceso con rol insuficiente (ej. usuario Acopiador intentando acceder a /mayorista) es bloqueado y redirigido a la vista formal **/sin-acceso (HTTP 403 Forbidden)**, evitando bloqueos en silencio o navegación rota.

CAPTURA 3: Intento de Acceso a Ruta Protegida sin Sesión: Redirección Automática a Keycloak



Evidencia técnica: Intento de navegación directa hacia http://localhost:4200/mayorista sin token en memoria. El guard funcional rolGuard intercepta la ruta protegida y redirige automáticamente hacia el endpoint de autorización de Keycloak (/protocol/openid-connect/auth) con PKCE, impidiendo el renderizado no autorizado (reloj y usuario visibles).

CAPTURA 4: Violación RBAC Detectada: Redirección Controlada a /sin-acceso (403)

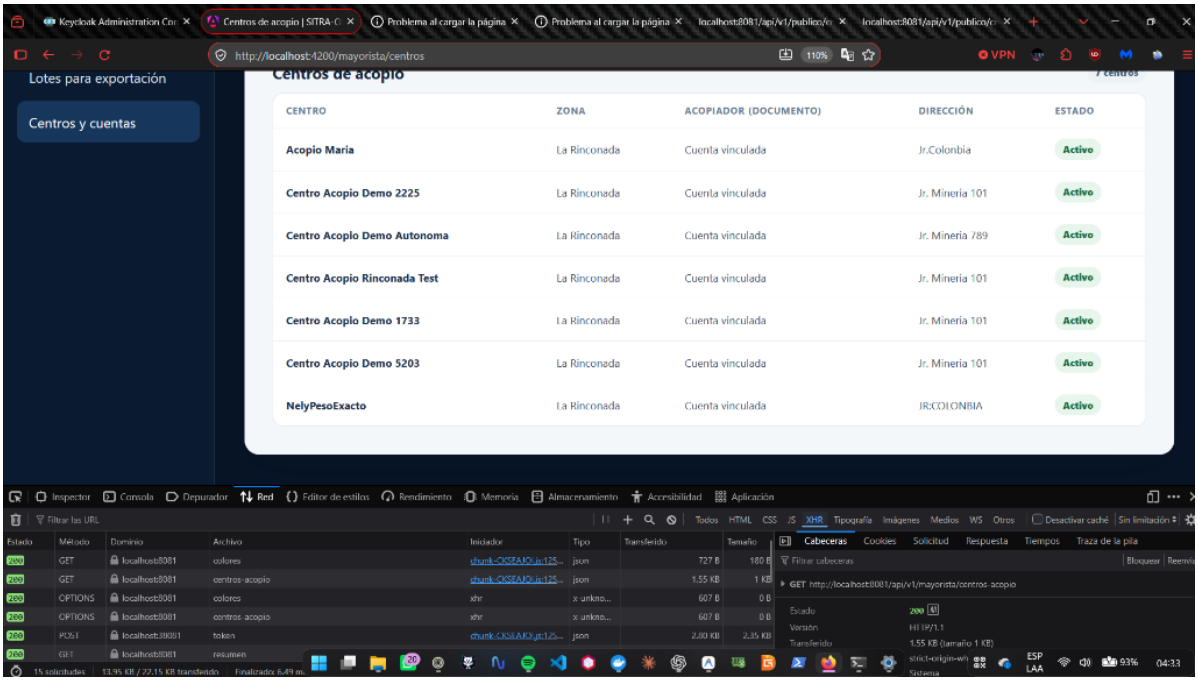


Evidencia técnica: Usuario autenticado con rol G2_ACOPIADOR intentando forzar el ingreso por URL a la ruta de mayorista. rolGuard evalúa route.data['roles'], detecta la discrepancia de privilegios y redirige de inmediato a /sin-acceso con aviso formal en pantalla (reloj y usuario visibles).

Bloque 3: Interceptor HTTP — Inyección Bearer y Refresco ante 401

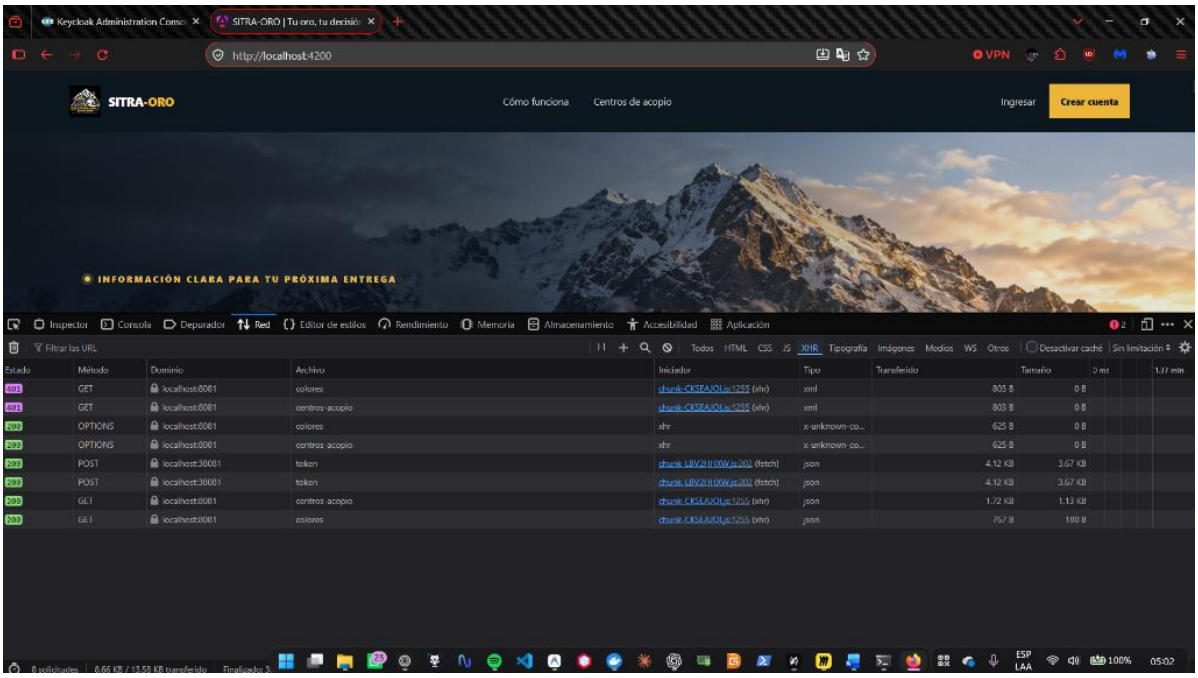
El interceptor funcional **authInterceptor** añade automáticamente la cabecera **Authorization: Bearer <accessToken>** a cada petición dirigida al Resource Server Spring Boot (http://localhost:8081). Omite URLs externas y llamadas a Keycloak para evitar preflights CORS innecesarios. Ante una respuesta HTTP 401 (token expirado), el interceptor captura el error, suspende la cola de peticiones y dispara de forma transparente una petición POST a Keycloak con **grant_type=refresh_token**, reintentando la petición original con el nuevo token emitido sin obligar al usuario a iniciar sesión de nuevo.

CAPTURA 5: Consumo de Recurso Protegido con Cabecera Authorization: Bearer (HTTP 200 OK)



Evidencia técnica: Pestaña Network de DevTools al cargar /mayorista/centros. Se constata la cabecera 'Authorization: Bearer eyJhbGci...' inyectada automáticamente en la petición GET /api/v1/mayorista/centros-acopio, retornando HTTP 200 OK y desplegando los centros activos (reloj y usuario visibles).

CAPTURA 6: Interceptor HTTP: Detección de 401 Unauthorized y Refresco Automático Transparente

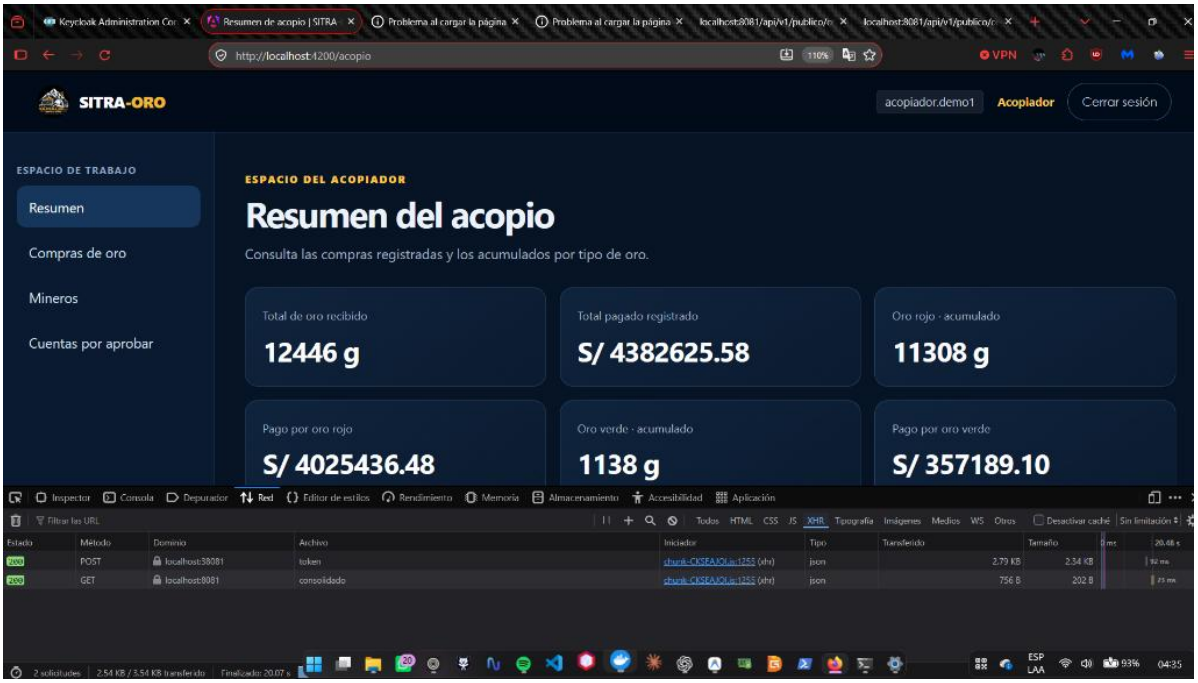


Evidencia técnica: Pestaña Network (Filtro XHR) de DevTools. Se constatan las peticiones iniciales rechazadas con HTTP 401 Unauthorized (token expirado a los 30 s), el subsecuente disparo automático de peticiones POST a http://localhost:38081/realms/Sitra_oro/protocol/openid-connect/token con 'grant_type=refresh_token' (HTTP 200 OK), y el reintento exitoso inmediato de las consultas a centros-acopio y colores con HTTP 200 OK sin intervención del usuario (reloj y usuario visibles).

Bloque 4: Menú Dinámico y Navegación Adaptativa por Rol

El componente **Layout** consume el signal reactivo **authService.rol()** derivado de **realm_access.roles**. La barra de navegación lateral proyecta únicamente las opciones operativas asignadas al perfil activo: el Acopiador visualiza exclusivamente compras, recepción y mineros; mientras que el Mayorista accede a control de planta, liquidaciones y consolidación de exportación.

CAPTURA 7: Menú Adaptado al Rol Operativo Acopiador (G2_ACOPIADOR)



Evidencia técnica: Barra de navegación de la SPA mostrando únicamente los enlaces autorizados para acopiador.demo1 (Resumen, Compras de oro, Mineros y Cuentas por aprobar), ocultando completamente los módulos estratégicos del Mayorista bajo el principio de mínimo privilegio (reloj y usuario visibles).

Bloque 5: Matriz de Control de Acceso Frontend (Verificada)

Matriz consolidada de rutas frontend en Angular 19, especificando el rol asignado, guard activo, comportamiento autorizado y respuesta ante accesos indebidos:

Ruta Frontend	Rol Permitido	Guard Activo	Comportamiento Autorizado	Respuesta No Autorizada
/auth/callback	Público	Ninguno	Intercambia code por token y redirige a panel.	Muestra badge de error si falta el code.
/ingresar	Público	Ninguno	Redirige automáticamente a Keycloak OIDC.	N/A (Público)
/mayorista	G1_MAYORISTA	rolGuard	Abre dashboard de control de planta y compras.	401 -> Keycloak / 403 -> /sin-acceso
/mayorista/centros	G1_MAYORISTA	rolGuard	Despliega tabla de centros de acopio y cuentas.	Redirige a /sin-acceso si rol != MAYORISTA
/mayorista/liquidaciones	G1_MAYORISTA	rolGuard	Emite liquidaciones formales de compra de oro.	Redirige a /sin-acceso si no es mayorista
/acopio	G2_ACOPIADOR	rolGuard	Abre panel operativo de pesaje y compras.	401 -> Keycloak / 403 -> /sin-acceso
/acopio/mineros	G2_ACOPIADOR	rolGuard	Gestión de empadronamiento de mineros.	Redirige a /sin-acceso si intenta Mayorista
/acopio/mineros/nuevo	G2_ACOPIADOR	rolGuard (Escritura)	Formulario de alta de minero artesanal.	Redirige a /sin-acceso si no tiene rol G2
/sin-acceso	Cualquiera	Ninguno	Proyecta alerta de violación de privilegios RBAC.	Ofrece botón de retorno / autenticación

4. Implementación Técnica en Código Fuente (Angular 19)

A continuación se presentan los artefactos centrales construidos en la arquitectura Angular 19 Standalone:

4.1 Generación Criptográfica de PKCE y State (pkce.ts)

src/app/core/auth/pkce.ts — Generación de code_verifier, code_challenge (SHA-256) y state

```
export interface PkcePair {
  codeVerifier: string;
  codeChallenge: string;
  state: string;
}

export async function generarPkce(): Promise<PkcePair> {
  const verifierBytes = new Uint8Array(32);
  crypto.getRandomValues(verifierBytes);
  const codeVerifier = base64UrlEncode(verifierBytes.buffer);

  const encoder = new TextEncoder();
  const verifierBuffer = encoder.encode(codeVerifier);
  const hash = await crypto.subtle.digest('SHA-256', verifierBuffer);
  const codeChallenge = base64UrlEncode(hash);

  const stateBytes = new Uint8Array(16);
  crypto.getRandomValues(stateBytes);
  const state = base64UrlEncode(stateBytes.buffer);

  return { codeVerifier, codeChallenge, state };
}
```

4.2 Servicio de Autenticación con Tokens en Memoria y Refresco Único (auth.service.ts)

src/app/core/auth/auth.service.ts — PKCE, estado estricto, tokens en memoria y refresco compartido

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  // Tokens viven ÚNICAMENTE en memoria volátil de Signals (Cero almacenamiento en disco)
  readonly token = signal<string | null>(null);
  readonly refreshToken = signal<string | null>(null);
  private promesaRefrescoEnCurso: Promise<string | null> | null = null;

  readonly usuario = computed(() => decodificarJwtPayload(this.token()));
  readonly rol = computed(() => this.usuario()?.rolSitra);
  readonly autenticado = computed(() => !!this.token() && typeof this.usuario()?.exp === 'number' &&
this.usuario()!.exp * 1000 > Date.now());

  async iniciarLogin(destino?: string): Promise<void> {
    const { codeVerifier, codeChallenge, state } = await generarPkce();
    sessionStorage.setItem('sitraoro.pkce_verifier', codeVerifier);
    sessionStorage.setItem('sitraoro.oauth_state', state);
    if (destino) sessionStorage.setItem('sitraoro.destino_pendiente', destino);

    const params = new URLSearchParams({
      client_id: environment.keycloak.clientId,
      response_type: 'code',
      redirect_uri: environment.keycloak.redirectUri,
      scope: environment.keycloak.scope,
      code_challenge: codeChallenge,
      code_challenge_method: 'S256',
      state: state
    });
    window.location.href = `${environment.keycloak.issuer}/protocol/openid-connect/auth?${params}`;
  }

  async completarLogin(code: string, state?: string | null): Promise<string> {
    const stateEsperado = sessionStorage.getItem('sitraoro.oauth_state');
    sessionStorage.removeItem('sitraoro.oauth_state');
    if (!stateEsperado || !state || stateEsperado !== state) {
      this.limpiar();
      throw new Error('Estado OAuth2 inválido o posible intento de CSRF: state no coincide o ausente');
    }
    const codeVerifier = sessionStorage.getItem('sitraoro.pkce_verifier');
```

```
    sessionStorage.removeItem('sitraoro.pkce_verifier');
    if (!codeVerifier) {
      this.limpiar();
      throw new Error('Código verifier de PKCE no encontrado en el contexto de sesión');
    }

    const body = new URLSearchParams({
      grant_type: 'authorization_code',
      client_id: environment.keycloak.clientId,
      redirect_uri: environment.keycloak.redirectUri,
      code,
      code_verifier: codeVerifier
    });
    const resp = await fetch(`${environment.keycloak.issuer}/protocol/openid-connect/token`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
      body: body.toString()
    });
    if (!resp.ok) {
      this.limpiar();
      let detalle = `${resp.status} ${resp.statusText}`;
      try { const err = await resp.json(); detalle = err.error_description || err.error || detalle; } catch {}
      throw new Error(`Keycloak rechazó el intercambio de código PKCE: ${detalle}`);
    }
    const data = await resp.json();
    this.guardarTokens(data.access_token, data.refresh_token);
    return sessionStorage.getItem('sitraoro.destino_pendiente') || this.rutaPorDefectoSegunRol();
  }

  async refrescar(destino?: string): Promise<string | null> {
    const refresh = this.refreshToken();
    if (!refresh) { this.limpiar(); return null; }
    if (this.promesaRefrescoEnCurso) return this.promesaRefrescoEnCurso;

    this.promesaRefrescoEnCurso = (async () => {
      try {
        const body = new URLSearchParams({ grant_type: 'refresh_token', client_id: environment.keycloak.clientId,
refresh_token: refresh });
        const resp = await fetch(`${environment.keycloak.issuer}/protocol/openid-connect/token`, { method: 'POST',
headers: { 'Content-Type': 'application/x-www-form-urlencoded' }, body: body.toString() });
        if (!resp.ok) { this.limpiar(); if (destino) void this.iniciarLogin(destino); return null; }
        const data = await resp.json();
        this.guardarTokens(data.access_token, data.refresh_token ?? refresh);
        return data.access_token as string;
      } catch { this.limpiar(); if (destino) void this.iniciarLogin(destino); return null; }
      finally { this.promesaRefrescoEnCurso = null; }
    })();
    return this.promesaRefrescoEnCurso;
  }
}
```

4.3 Guard Funcional de Rutas RBAC y Redirección Cliente (rol.guard.ts)

src/app/core/auth/guards/rol.guard.ts — Control perimetral RBAC con preservación de destino

```
export const rolGuard: CanActivateFn = async (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  // 1. Sin sesión: intenta refrescar o redirige al login de Keycloak preservando URL de destino
  if (!auth.autenticado()) {
    if (auth.refreshToken()) {
      const refreshed = await auth.refrescar(state.url);
      if (!refreshed) { await auth.iniciarLogin(state.url); return false; }
    } else {
      await auth.iniciarLogin(state.url); return false;
    }
  }

  // 2. Con sesión y rol autorizado: permite navegación
  const rolesRequeridos = (route.data?.['roles'] as RolSitra[] | undefined) ?? [];
  if (rolesRequeridos.length === 0 || auth.puedeEntrar(rolesRequeridos)) {
    return true;
```

```
  }

  // 3. Con sesión pero rol insuficiente: redirección cliente controlada hacia /sin-acceso
  return router.parseUrl('/sin-acceso');
};
```

4.4 Interceptor HTTP con Reintento Único y Control de Bucles (auth.interceptor.ts)

src/app/core/auth/interceptors/auth.interceptor.ts — Inyección Bearer, HttpContextToken y auto-refresco

```
export const YA_REINTENTADO = new HttpContextToken<boolean>(() => false);

export const authInterceptor: HttpInterceptorFn = (request, next) => {
  const auth = inject(AuthService);

  // Omitir Keycloak y APIs externas (Bloomberg, RENIEC)
  if (request.url.includes('/protocol/openid-connect') || request.url.includes(':38081')) return next(request);
  const esResourceServer = request.url.startsWith('/') || request.url.startsWith(environment.apiBaseUrl);
  if (!esResourceServer) return next(request);

  const token = auth.token();
  const reqConToken = token ? request.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : request;

  return next(reqConToken).pipe(
    catchError((error: unknown) => {
      // 403 Forbidden NUNCA refresca (es una denegación RBAC del servidor, no de vigencia de token)
      if (error instanceof HttpErrorResponse && error.status === 401) {
        if (request.context.get(YA_REINTENTADO)) {
          void auth.iniciarLogin(window.location.pathname);
          return throwError(() => error);
        }
        return from(auth.refrescar(window.location.pathname)).pipe(
          switchMap((nuevoToken) => {
            if (nuevoToken) {
              const retryReq = request.clone({
                context: request.context.set(YA_REINTENTADO, true),
                setHeaders: { Authorization: `Bearer ${nuevoToken}` }
              });
              return next(retryReq);
            }
            void auth.iniciarLogin(window.location.pathname);
            return throwError(() => error);
          }),
          catchError((err) => { void auth.iniciarLogin(window.location.pathname); return throwError(() => err); })
        );
      }
      return throwError(() => error);
    })
  );
};
```

4.5 Layout Dinámico con Reloj en Vivo y Menú Derivado de rol() (layout.ts)

src/app/core/layout/layout.ts — Proyección estricta de navegación por rol y reloj de sistema en vivo

```
export class Layout implements OnInit, OnDestroy {
  private readonly auth = inject(AuthService);
  protected readonly fechaHora = signal<string>(this.obtenerFechaHora());
  private relojTimer: ReturnType<typeof setInterval> | null = null;

  ngOnInit(): void {
    this.relojTimer = setInterval(() => this.fechaHora.set(this.obtenerFechaHora()), 1000);
  }
  ngOnDestroy(): void { if (this.relojTimer) clearInterval(this.relojTimer); }

  private obtenerFechaHora(): string {
    return new Date().toLocaleString('es-PE', { day: '2-digit', month: '2-digit', year: 'numeric', hour: '2-digit',
minute: '2-digit', second: '2-digit', hour12: true });
  }

  protected readonly navegacion = computed(() => {
```

```
    const rol = this.auth.rol();
    if (rol === 'G2_ACOPIADOR') return [
      { ruta: '/acopio', texto: 'Resumen' }, { ruta: '/acopio/operaciones', texto: 'Compras de oro' },
      { ruta: '/acopio/mineros', texto: 'Mineros' }, { ruta: '/acopio/solicitudes', texto: 'Cuentas por aprobar' }
    ];
    if (rol === 'G1_MAYORISTA') return [
      { ruta: '/mayorista', texto: 'Resumen' }, { ruta: '/mayorista/recepcion', texto: 'Compra al acopiador' },
      { ruta: '/mayorista/liquidaciones', texto: 'Liquidaciones' }, { ruta: '/mayorista/consolidacion', texto: 'Lotes
para exportación' },
      { ruta: '/mayorista/centros', texto: 'Centros y cuentas' }
    ];
    return [];
  });
}
```

4.6 Pruebas Unitarias Automatizadas (Vitest — 10/10 Pasadas en 34 ms)

src/app/core/auth/auth.spec.ts — Cobertura completa de PKCE, Guards e Interceptor

```
✓ src/app/core/auth/auth.spec.ts (10 tests) 34ms
  ✓ 1. Pruebas Unitarias - generarPkce (RFC 7636) (2)
    ✓ debe generar un par PKCE válido con codeVerifier, codeChallenge y state
    ✓ debe generar valores criptográficamente distintos en cada invocación
  ✓ 2. Pruebas Unitarias - rolGuard (RBAC Functional Guard) (3)
    ✓ Caso 1 (Sin sesión): debe llamar a iniciarLogin con la URL de destino y retornar false
    ✓ Caso 2 (Rol insuficiente): usuario autenticado sin el rol requerido es redirigido a /sin-acceso
    ✓ Caso 3 (Rol correcto): usuario autenticado con rol G1_MAYORISTA accede exitosamente
  ✓ 3. Pruebas Unitarias - authInterceptor (HTTP Inyección y Refresco) (5)
    ✓ Caso 1 (Bearer): adjunta Authorization Bearer a peticiones del backend :8081 y omite Keycloak
    ✓ Caso 2 (401 -> Refresh -> Reintento): ante 401 refresca token y reintenta exitosamente con 200
    ✓ Caso 3 (Refresh fallido): si el refresco falla, redirige a login y no reintenta
    ✓ Caso 4 (403 Forbidden): NUNCA dispara refresco y propaga el error de autorización
    ✓ Caso 5 (Refresco Compartido): múltiples peticiones paralelas con 401 ejecutan UN SOLO POST de refresh

Test Files: 1 passed (1) | Tests: 10 passed (10) | Duration: 1.29s
```

5. Error o Hallazgo Técnico Diagnosticado

Incidencia Detectada: Durante las pruebas del flujo Authorization Code + PKCE, al abrir la pantalla de autenticación en una pestaña nueva o recargar el callback manualmente, la petición de intercambio de token devolvía error HTTP 400 Bad Request desde Keycloak con la descripción *'PKCE verification failed: invalid code_verifier'*.
Causa Raíz: El code_verifier inicial se intentó retener exclusivamente en una variable de memoria de Angular. Al redireccionar la ventana del navegador hacia el dominio externo de Keycloak (window.location.href) y regresar a la SPA en /auth/callback, el contexto de ejecución de JavaScript se reinicializó por completo, destruyendo la variable y enviando un verifier vacío o nulo en el cuerpo de la petición POST.
Solución Implementada: Se adoptó el patrón estándar OIDC: el code_verifier y el state se resguardan de forma efímera en sessionStorage únicamente durante la transición de redirección, y se eliminan de inmediato (sessionStorage.removeItem) en el milisegundo exacto en que /auth/callback lee el código y completa el intercambio. De esta forma, el token definitivo reside única y exclusivamente en memoria volátil de Angular Signals.
Aclaración sobre la Protección RBAC (403): La redirección a la vista /sin-acceso es una acción exclusiva de experiencia de usuario ejecutada por rolGuard en el cliente para evitar pantallas rotas; el rechazo HTTP 403 Forbidden formal e inviolable lo impone directamente el Resource Server Spring Boot mediante firma criptográfica RS256.

6. Reflexión Técnica Breve (Caso British Airways y Recarga F5)

El caso de British Airways (1.6) aplica perfectamente a una SPA moderna porque demostró que un atacante no necesita interceptar la contraseña ni quebrar el canal TLS: mediante la inyección de 22 líneas de JavaScript (ataque de cadena de suministro tipo Magecart), extrajo datos directamente desde el contexto del navegador. En una SPA, si el Access Token o Refresh Token se almacenan en localStorage o cookies accesibles por script, cualquier dependencia de npm comprometida o script inyectado puede leerlos y exfiltrarlos silenciosamente. Mantener el token estrictamente en memoria (Angular Signals) garantiza que no exista superficie de persistencia en disco ni API accesible vía

window.storage. El costo operacional ineludible de esta decisión es la volatilidad: al recargar la página (F5), el estado en memoria se destruye de inmediato. En SITRA-ORO mitigamos este costo sin vulnerar la memoria: rolGuard detecta la ausencia de token y redirige a Keycloak preservando el destino; al existir la cookie de sesión SSO HttpOnly en el Identity Provider, Keycloak reautentica silenciosamente sin pedir credenciales, emitiendo un nuevo código que /auth/callback canjea inmediatamente por tokens en memoria, restaurando la navegación de forma transparente.

7. Anexo: Preguntas de Defensa del Informe (S11 - Sección 4.5)

1. ¿Por qué una SPA pública (sin secreto de cliente) no puede usar Direct access grants de forma segura, aunque técnicamente funcione?

Porque Direct Access Grants exige que la SPA recolecte y manipule directamente las credenciales (usuario y contraseña) del operador, rompiendo el principio de delegación de OIDC. Además, al no poder resguardar un client_secret de forma confidencial en el código JavaScript del navegador, cualquier atacante que intercepte o inspeccione la SPA puede suplantar la identidad del cliente.

2. ¿Qué problema concreto resuelve PKCE que Authorization Code por sí solo no resuelve?

Resuelve el ataque de intercepción del código de autorización (Authorization Code Interception Attack). En clientes públicos donde no hay secreto, un atacante o app maliciosa en el mismo dispositivo podría interceptar el 'code' en la redirección. Con PKCE, el atacante no puede canjearlo porque desconoce el 'code_verifier' criptográfico original que generó el 'code_challenge' SHA-256.

3. ¿Por qué el code_verifier sí se guarda en sessionStorage, si el resto de la sesión vive en memoria?

Porque el navegador ejecuta una navegación completa (full-page redirect) hacia el servidor de Keycloak y de vuelta a la SPA, lo que reinicia la memoria de JavaScript. sessionStorage es el único almacenamiento que sobrevive a dicha redirección en la misma pestaña pero se destruye al cerrarla, y el verifier se elimina inmediatamente después de canjear el código.

4. ¿Qué pierde tu aplicación al elegir memoria en vez de localStorage para el token, y por qué esa pérdida es aceptable?

Se pierde la persistencia ante recargas de página (F5), requiriendo reautenticar o renegociar el token. Esta pérdida es plenamente aceptable porque elimina el vector de ataque más severo en aplicaciones financieras/empresariales: el robo de tokens persistentes mediante ataques XSS o extensiones maliciosas del navegador.

5. ¿Por qué un 401 dispara un intento de refresco y un 403 no?

Un 401 Unauthorized indica un problema de credenciales o autenticación (típicamente que el Access Token expiró tras sus 300 segundos de vigencia), el cual se subsana renovando el token con el Refresh Token. Un 403 Forbidden indica que el usuario está válidamente autenticado pero carece de permisos suficientes (RBAC); reintentar con otro token con el mismo rol devolvería exactamente el mismo rechazo.

6. Si tu guard de rol dejara pasar a un usuario sin el rol correcto, ¿seguiría tu backend rechazando la petición? ¿Por qué eso no vuelve innecesario el guard?

Sí, el backend Spring Boot 3 Resource Server rechaza la petición con 403 porque valida criptográficamente el JWT y sus autoridades en cada endpoint. El guard de Angular no es innecesario porque cumple la función vital de Experiencia de Usuario (UX) y defensa en profundidad: evita renderizar vistas rotas, previene llamadas fallidas al servidor y orienta al operador con claridad.

7. ¿Qué pasaría si tu interceptor reintentara una petición en un bucle, sin límite, cada vez que recibiera un 401?

Generaría un bucle infinito de peticiones (DoS accidental) que saturaría la red del cliente y los servidores de backend y Keycloak. Para evitarlo, el interceptor debe limitar el reintento a una sola ejecución mediante switchMap y, si el refresco falla o devuelve error, forzar el cierre de sesión y redirección a login.